



Editorial

Rapid Continuous Software Engineering



This is the 2nd special issue focusing on the challenges and issues related to software engineering for processes supporting rapid continuous software engineering approaches. These are mainly areas where the overall approach can be characterized as releasing quickly and as often as possible to address the users' need in a much better way than the more classical style of "top down" development. There are, of course, areas of software development where components and systems need certification e.g. aerospace, train and automotive and the related regulations differ from country to country. But even in such cases, software systems contain parts and subsystems which can best be developed through including the user in a structured and systematic way during the development on a continuous basis.

The call for this issue sought to solicit papers on recent research results which could help advance knowledge as to how traditional software engineering aspects can be translated into ways of working in the rapid continuous development mode. In total 14 submissions were received, and following a stringent review process, four papers were accepted for eventual publication. These papers are briefly discussed below. All accepted contributions describe and evaluate approaches which empower developers and users in these agile scenarios as they sought to address the processes with respect to their speed and potential information overflow. Thus, the need to better understand and hence provide the ability to control the development processes is addressed in the contributions contained in this special issue:

"Going with the flow: An activity theory analysis of flow techniques in software development" by Dennehy and Conboy presents an empirical study of flow activities in two companies which are in the process of adopting flow techniques. The empirical study is based on activity theory to study multiple, not singular, activities and investigates contradictions and congruencies when people interact and use artifacts. Hence, the authors investigate the motivation of the companies to adopt those techniques as well as major contradictions and congruencies in their implementation and use. They identified four key contradictions, including that flow practices provide greater visibility into what teams are doing but can also lead to the perception by software engineering teams that management will use the tools to control developers.

Roles in software engineering teams and projects differ between classical software engineering processes and agile ones. The paper "Transition of organizational roles in Agile transformation process: A grounded theory approach" by Jovanovic et al. investigates how those roles change when companies introduce agile software engineering processes. Based on interviews as well as observations, the authors identify multiple aspects of inadequate transformations to agile roles, e.g., causes, conditions, contingencies,

consequences. While the focus is on roles, the paper also presents general challenges and solutions with respect to agile transformation.

Continuous experimentation aims to reduce development risks by iteratively testing assumptions that are critical to the success of the software. In their paper "Introducing continuous experimentation in large software-intensive product and service organisations", Yaman et al. investigate how continuous experimentation is introduced in development processes using a multiple case study on two Finnish companies. Identified benefits include better customer insight and reduced development effort. Challenges are, for example, the lack of experimentation skills and knowledge, as well as difficulties to access end users.

User acceptance testing is an important activity in classical and agile software engineering projects. In their paper "User acceptance testing for Agile-developed web-based applications: Empowering customers through wikis and mind maps", Otaduy and Diaz present an approach using wikis and mind maps to empower customers to perform user acceptance testing on their own. Wiki pages are created by the developers to support acceptance testing by customers and include starting pages for tests as well as testing hints. Additionally, they store executable acceptance test scripts defined by the user. As users lack the expertise to write these test scripts, the approach contains a mind-map-based graphical domain specific language for the modeling of acceptance tests.

Last but not least, we would also like to thank the authors of all submissions and the reviewers for their dedication and commitment in providing a rigorous review process enabling this publication. Without their efforts, the software engineering community would not have this important opportunity to study the disruptive changes that arise through the adoption of rapid continuous software engineering development processes.

Guest Editors

Matthias Tichy
University of Ulm, Germany

Michael Goedicke*
paluno, University of Duisburg-Essen, Germany

Jan Bosch
Chalmers University of Technology, Sweden

Brian Fitzgerald
Lero, University of Limerick, Ireland

*Corresponding author.

E-mail address: Michael.Goedicke@paluno.uni-due.de (M. Goedicke)